INTERNSHIELD

LEARN • PRACTICE • DEFEND • SECURE • INNOVATE

CLOUD COMPUTING & SECURITY

LAB 04

DOCKER IMAGE VULNERABILITY ASSESSMENT

Scan and analyze Docker images to identify vulnerabilities, misconfigurations and security risks.

SCAN
Scan Docker images for OS and application vulnerabilities

DETECT
Detect known CVEs and security misconfigurations

ANALYZE
Analyze severity and understand the associated risks

REMEDIATE
Apply best practices and update base images

SECURE
Strengthen container security and maintain a secure pipeline

PRACTICAL LABORATORY MANUAL

HANDS-ON CLOUD SECURITY EXERCISES

AUTHORIZED TRAINING ENVIRONMENT

This lab is designed for educational purposes only. Perform all exercises only in authorized lab environments and with explicit permission.

VIDIT SHRINGI | KRITI PUROHIT | EDITION 2026 | LINKEDIN https://www.linkedin.com/company/internshield.in

LAB INFORMATION

FIELD	DETAIL
LAB	4
TITLE	Perform Vulnerability Assessment on Docker Images
PRIMARY PLATFORM	Docker Container Runtime
SERVICE	Docker Images / Container Registry
PRIMARY TOOL	Trivy
LAB TYPE	Container Security Assessment
FOCUS	Docker Image Vulnerability Scanning

LAB SCENARIO

As a professional ethical hacker or penetration tester, expertise in Docker vulnerability assessment is essential. By leveraging tools such as Trivy, an assessor can analyze Docker images to identify vulnerabilities and misconfigurations before they can be leveraged by an attacker.

Active scanning and manual inspection reveal weak configurations and outdated packages that could otherwise be used to breach a container's security boundary or introduce malicious code. Understanding where an image originates from, and what it contains, is a key part of comprehensive container security testing and mitigation.

AUTHORIZED LAB ENVIRONMENT

This exercise must be performed only against the images and targets provided or authorized for the training environment.

Never scan or pull images from unauthorized private registries, and never use assessment findings to attack production containers without explicit authorization.

LAB OBJECTIVES

- Perform vulnerability assessment on Docker images using Trivy

The learner will understand how a container security scanner identifies known vulnerabilities in image layers, and why base-image hygiene is a core part of the container-security lifecycle.

OVERVIEW OF DOCKER IMAGES

Docker images are lightweight, standalone, executable packages that contain everything needed to run a software application, including the code, runtime, libraries, and dependencies. They enable consistent deployment across different environments, simplify software distribution, and support scalability and reproducibility in containerized environments.

TOOL OVERVIEW

Trivy

Trivy is a security scanner that detects vulnerabilities and misconfigurations across a wide range of targets, including container images, file systems, Git repositories, virtual machine images, Kubernetes, and AWS. With its comprehensive scanners, Trivy identifies OS package vulnerabilities, sensitive information, infrastructure-as-code issues, and more, providing a robust security solution for container-based infrastructure.

DOCKER IMAGE	TRIVY SCAN ENGINE	OS PACKAGE DB	CVE MATCHING	VULNERABILITY REPORT

LAB ENVIRONMENT & PREREQUISITES

Requirement	Purpose
Parrot Security	Security-testing workstation
MATE Terminal	Command-line interface
Docker	Container runtime used to pull and run images
Trivy	Container image vulnerability scanner
Internet Connectivity	Required to pull images and vulnerability database updates

SECURITY & AUTHORIZATION WARNING

Vulnerability scanning reveals detailed information about the software packages contained in an image, including known CVEs.

Perform this laboratory ONLY against:

- Publicly published base images used strictly for training (as in this lab)
- Instructor-provided or organization-approved images

Do not use scan findings to attack systems you are not authorized to test.

TASK 1

VULNERABILITY ASSESSMENT ON DOCKER IMAGES USING TRIVY

OBJECTIVE

Use Trivy to scan Docker images and compare the vulnerability posture of a well-maintained image against an outdated, vulnerable image, then document the resulting findings for security assessment.

PRACTICAL PROCEDURE

STEP 01 LAUNCH THE TERMINAL

ACTION

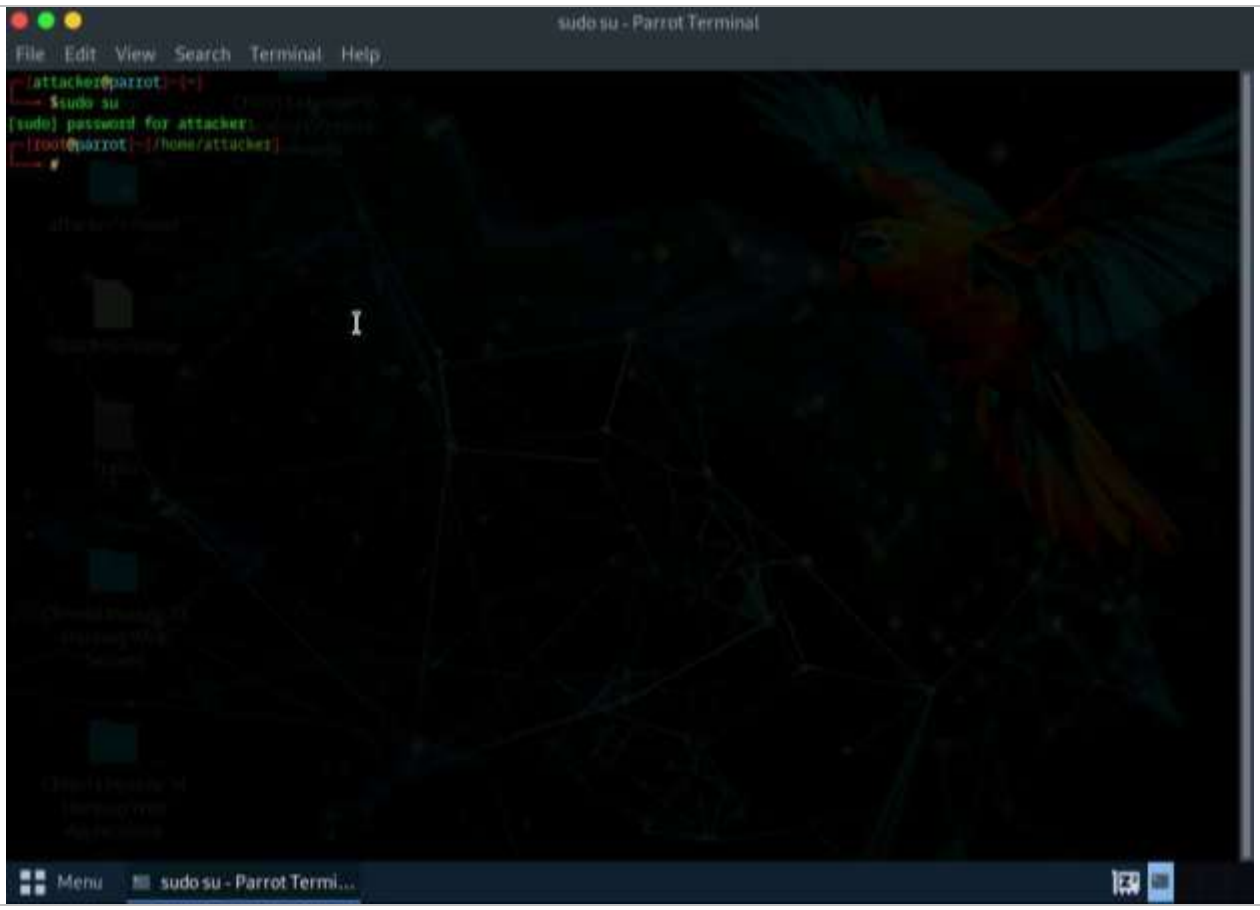
In the Parrot Security machine, click the MATE Terminal icon in the menu to launch the terminal.

EXPECTED RESULT

A Terminal window opens.

SECURITY INSIGHT

Working from the dedicated security workstation keeps the assessment environment isolated from production systems.



STEP 02 OPEN A ROOT SHELL

ACTION

A Parrot Terminal window appears. In the terminal window, type sudo su and press Enter to run the programs as a root user.

```
sudo su
```

EXPECTED RESULT

The shell prompt reflects root-level access.

SECURITY INSIGHT

Docker and Trivy commands often require elevated privileges to interact with the container runtime; elevate only within the authorized lab session.

STEP 03 PROVIDE THE SUDO PASSWORD

ACTION

In the [sudo] password for attacker field, type toor as a password and press Enter. The password will not be visible as you type. Minimize the terminal afterward for a better view of the command output.

EXPECTED RESULT

Root access is granted in the terminal.

SECURITY INSIGHT

Password masking is a standard terminal security behavior that prevents shoulder-surfing of credentials.

STEP 04 PLAN THE COMPARATIVE SCAN

ACTION

In this lab, two Docker images will be scanned: first a well-maintained, secure image, and second a known outdated, vulnerable image, so that the results can be compared.

EXPECTED RESULT

The two-image comparison approach is understood before scanning begins.

SECURITY INSIGHT

Comparing a well-maintained image against a known-vulnerable one is an effective way to illustrate how base-image choice and patch currency directly affect exposure.

STEP 05 PULL THE FIRST DOCKER IMAGE

ACTION

Execute the command docker pull ubuntu to pull the first Docker image.

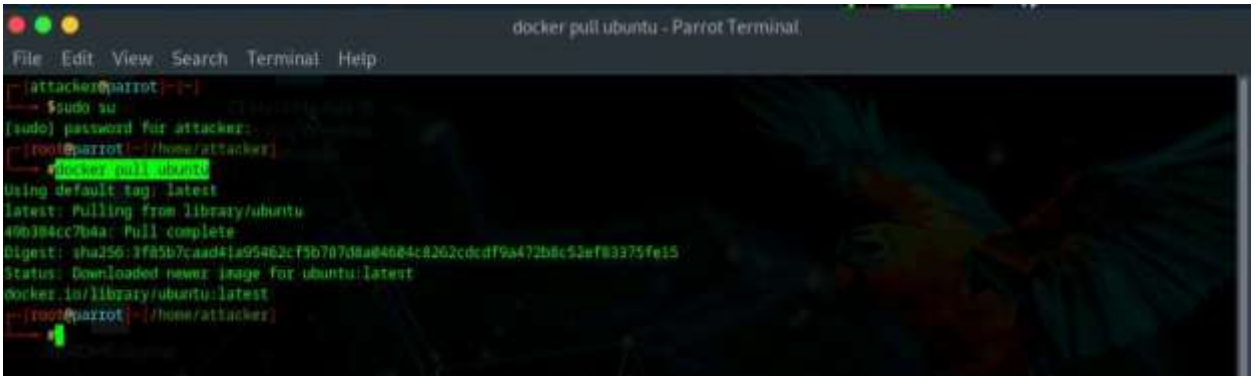
```
docker pull ubuntu
```

EXPECTED RESULT

The ubuntu image downloads successfully to the local Docker environment.

SECURITY INSIGHT

Pulling from an official, actively maintained repository is itself a security-relevant choice, since official images typically receive more frequent security updates.



STEP 06 SCAN THE FIRST IMAGE WITH TRIVY

ACTION

Once the image is pulled, perform the vulnerability assessment. Execute the command trivy image ubuntu.

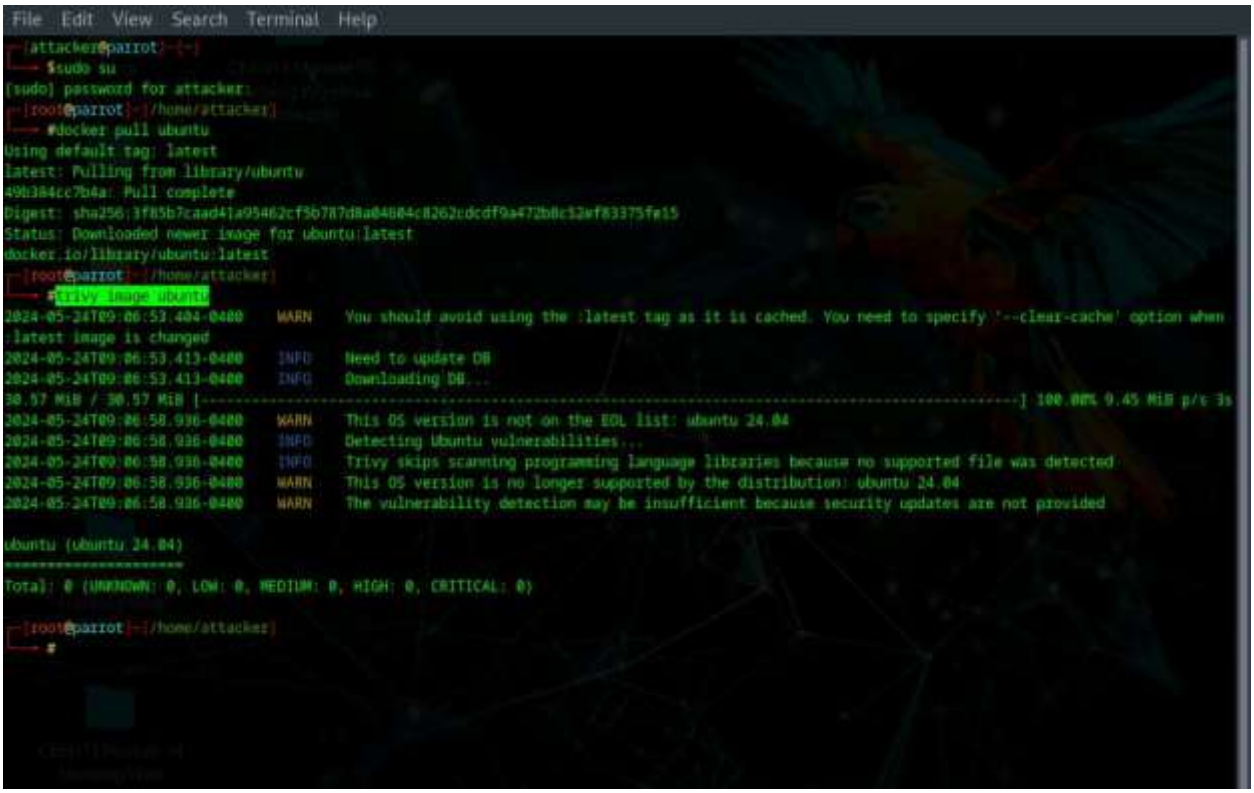
```
trivy image ubuntu
```

EXPECTED RESULT

Trivy scans the ubuntu image and returns a vulnerability report.

SECURITY INSIGHT

Trivy checks installed OS packages against known-vulnerability databases to surface CVEs relevant to that specific image build.



STEP 07 REVIEW THE FIRST SCAN RESULT

ACTION

Review the scan output. The result shows a total of 0 vulnerabilities, indicating the image is currently secure at the time of scanning.

EXPECTED RESULT

The ubuntu image is confirmed to have no detected vulnerabilities at scan time.

SECURITY INSIGHT

A clean scan result reflects the image's state at the moment it was pulled; images should be re-scanned periodically since new CVEs are disclosed continuously.

STEP 08 PULL THE SECOND DOCKER IMAGE

ACTION

Now analyze the vulnerable image. Execute the command docker pull nginx:1.19.6 to pull the outdated image.

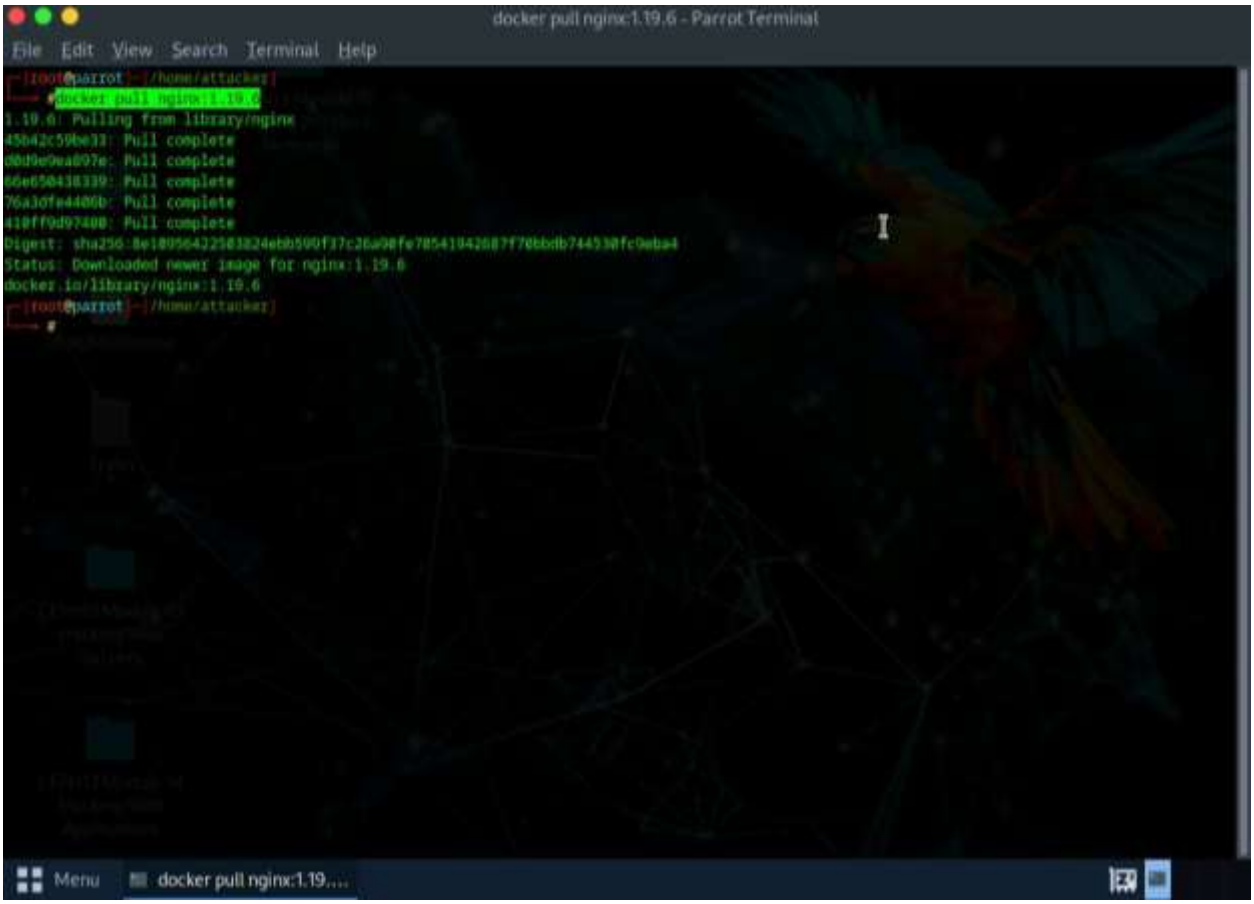
```
docker pull nginx:1.19.6
```

EXPECTED RESULT

The nginx:1.19.6 image downloads successfully to the local Docker environment.

SECURITY INSIGHT

Pinning to an older, specific version tag is a common real-world cause of accumulated unpatched vulnerabilities.



STEP 09 SCAN THE SECOND IMAGE WITH TRIVY

ACTION

Execute the command trivy image nginx:1.19.6 to scan the image.

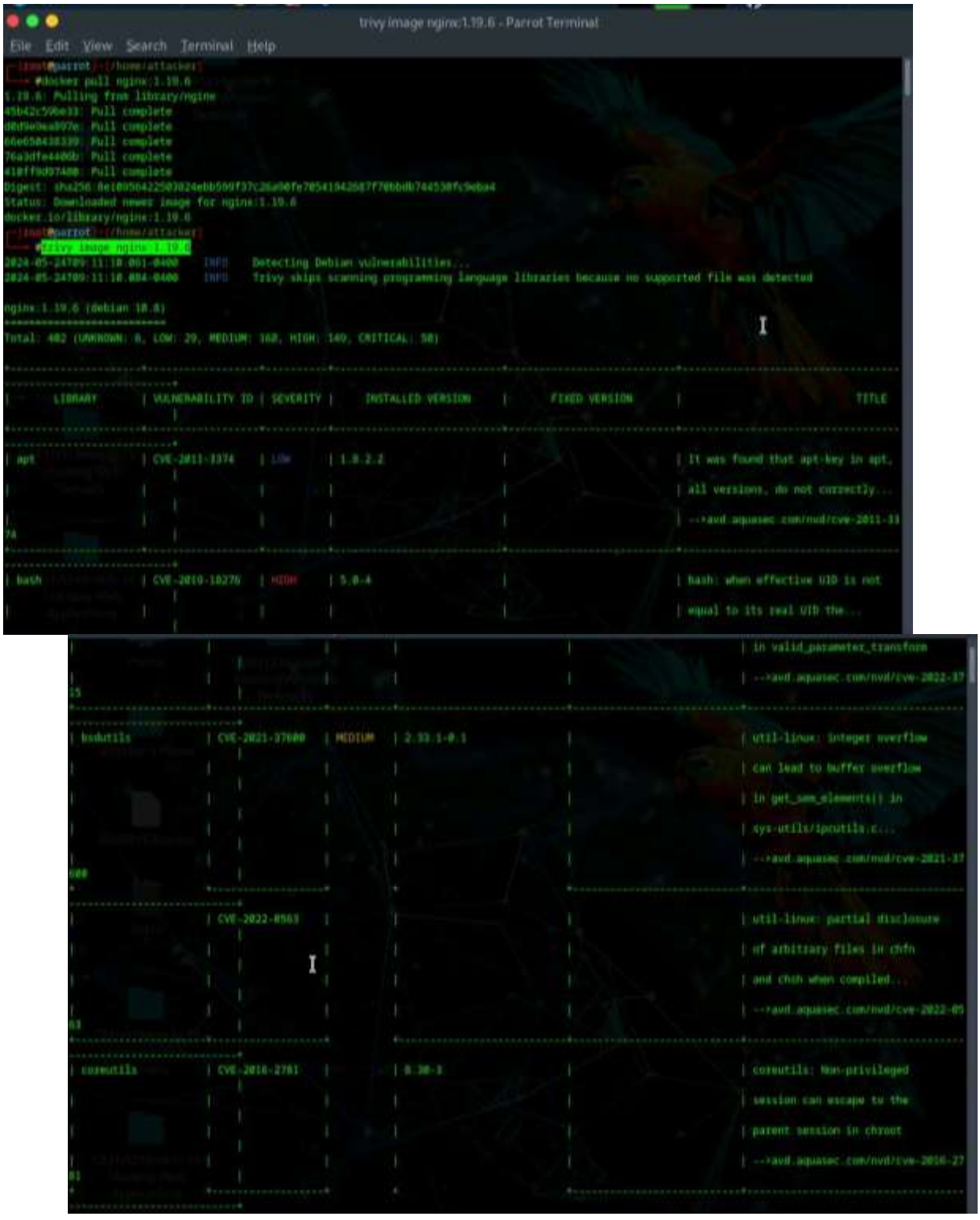
```
trivy image nginx:1.19.6
```

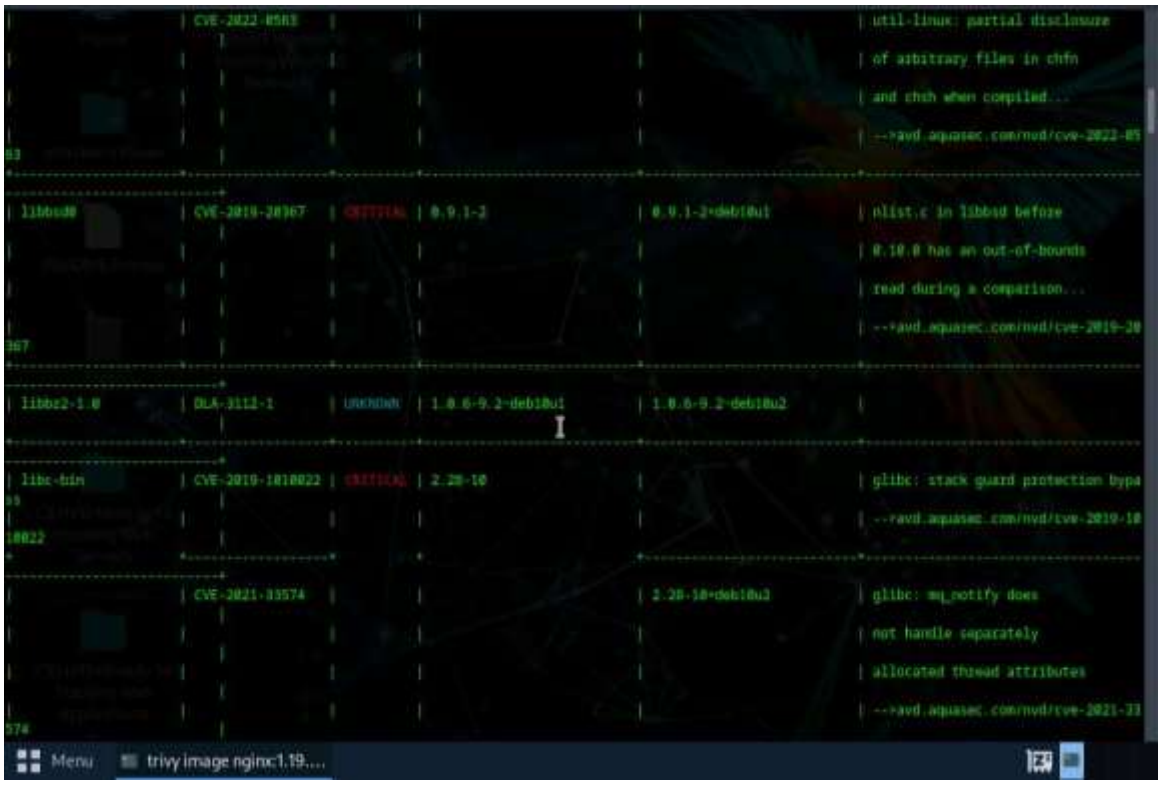
EXPECTED RESULT

Trivy scans the nginx:1.19.6 image and returns a detailed vulnerability report.

SECURITY INSIGHT

Scanning an older, unpatched image typically surfaces a much larger set of known CVEs than a current, actively maintained image.


trivy image nginx:1.19.6 - Parrot Terminal
File Edit View Search Terminal Help
#docker pull nginx:1.19.6
#trivy image nginx:1.19.6
Total: 402 (UNKNOWN: 0, LOW: 39, MEDIUM: 168, HIGH: 149, CRITICAL: 50)
LIBRARY | VULNERABILITY ID | SEVERITY | INSTALLED VERSION | FIXED VERSION | TITLE
apt | CVE-2011-3374 | LOW | 1.8.2.2 | | It was found that apt-key in apt, all versions, do not correctly...
bash | CVE-2019-18276 | HIGH | 5.0-4 | | bash: when effective UID is not equal to its real UID the...
| | | | | in valid_parameter_transform
| | | | | --> avd.aquasec.com/nvd/cve-2022-37
15
bsdutils | CVE-2021-37600 | MEDIUM | 2.33.1-0.1 | | util-linux: integer overflow can lead to buffer overflow in get_sem_elements() in sys-utils/ipcutils.c...
| | | | | --> avd.aquasec.com/nvd/cve-2021-37
600
| CVE-2022-0563 | | | | util-linux: partial disclosure of arbitrary files in chfn and chsh when compiled...
| | | | | --> avd.aquasec.com/nvd/cve-2022-05
63
coreutils | CVE-2016-2781 | | 8.30-3 | | coreutils: Non-privileged session can escape to the parent session in chroot
| | | | | --> avd.aquasec.com/nvd/cve-2016-27
81


CVE-2022-0563
util-linux: partial disclosure
of arbitrary files in chfn
and chsh when compiled...
--avd.aquasec.com/nvd/cve-2022-0563
libbsd0
CVE-2019-20367
CRITICAL
0.9.1-2
0.9.1-2+deb10u1
nlist.c in libbsd before
0.10.0 has an out-of-bounds
read during a comparison...
--avd.aquasec.com/nvd/cve-2019-20367
libbz2-1.0
DLA-3112-1
UNKNOWN
1.0.6-9.2~deb10u1
1.0.6-9.2~deb10u2
libc-bin
CVE-2019-1010022
CRITICAL
2.28-10
glibc: stack guard protection bypa
--avd.aquasec.com/nvd/cve-2019-1010022
CVE-2021-33574
2.28-10+deb10u2
glibc: mq_notify does
not handle separately
allocated thread attributes
--avd.aquasec.com/nvd/cve-2021-33574
Menu
trivy image nginx:1.19....

STEP 10 REVIEW THE SECOND SCAN RESULT

ACTION

Review the scan output. The result shows a total of 401 vulnerabilities, categorized by severity, with the corresponding CVEs listed for each finding.

EXPECTED RESULT

A total of 401 vulnerabilities are identified, categorized by severity and mapped to specific CVEs.

SECURITY INSIGHT

Severity categorization (e.g., critical, high, medium, low) helps prioritize remediation effort toward the findings with the greatest potential impact.

STEP 11 CONCLUDE THE SCANNING DEMONSTRATION

ACTION

This concludes the demonstration of vulnerability assessment on Docker images using Trivy.

EXPECTED RESULT

The comparative scanning demonstration is complete.

SECURITY INSIGHT

Formally closing out the technical activity helps transition cleanly into documentation and remediation planning.

STEP 12 CLOSE WINDOWS AND DOCUMENT FINDINGS

ACTION

Close all open windows and document all acquired information.

EXPECTED RESULT

Findings can be documented and the session is closed.

SECURITY INSIGHT

Thorough documentation of vulnerability findings, including severity and CVE references, is essential for follow-up remediation and reporting.

SECURITY ANALYSIS

Unpatched or outdated Docker images can carry known vulnerabilities forward into every container launched from them, creating risk across the environments where those containers run.

- Outdated base images — accumulate unpatched OS-level vulnerabilities over time
- Vulnerable dependencies — libraries and packages bundled into the image layer
- Known CVEs — publicly documented weaknesses that can be matched to specific package versions
- Severity distribution — critical and high-severity findings warrant the most urgent remediation
- Supply-chain risk — vulnerabilities inherited from third-party or community-maintained base images
- Configuration weaknesses — insecure defaults baked into the image at build time

DOCKER IMAGE SECURITY COUNTERMEASURES

- Use minimal, actively maintained base images.
- Regularly update and re-pull base images.
- Pin to current, supported version tags rather than outdated ones.
- Integrate image scanning into the CI/CD pipeline.
- Remediate critical and high-severity findings promptly.
- Use trusted, official registries where possible.
- Remove unnecessary packages and tools from images.
- Monitor newly disclosed CVEs affecting images in use.
- Apply least privilege to container runtime permissions.
- Document and track remediation of identified vulnerabilities.

IMAGE BUILD	VULNERABILITY SCAN	TRIAGE FINDINGS	PATCH / REBUILD	RE-SCAN & VERIFY

LEARNING OUTCOMES

After completing this lab, students should be able to:

- 1. Explain what a Docker image is and how it is composed.
- 2. Explain the purpose of Docker image vulnerability scanning.
- 3. Describe the purpose and capabilities of Trivy.
- 4. Pull Docker images from a registry using the Docker CLI.
- 5. Execute a Trivy scan against a Docker image.
- 6. Interpret Trivy scan output, including CVEs and severity levels.
- 7. Compare the vulnerability posture of different image versions.
- 8. Document container-security findings.
- 9. Recommend countermeasures to reduce image-level risk.

LAB OBSERVATION

Activity	Observation	Security Significance
Terminal and Root Access		
First Image Pull (ubuntu)		
First Image Scan Result		
Second Image Pull (nginx:1.19.6)		
Second Image Scan Result		
Severity Categorization		
CVE Review		
Final Documentation		

SECURITY FINDING

Finding	Risk	Impact	Countermeasure
ubuntu image — 0 vulnerabilities detected	Low (at scan time)	Minimal known exposure observed	Continue periodic re-scanning
nginx:1.19.6 image — 401 vulnerabilities detected	High	Multiple CVEs across categorized severities	Upgrade to a current, patched image version

Vulnerability counts reflect the state of each image's package database at the time it was scanned and will vary as new CVEs are disclosed or images are updated.

LAB COMPLETION CHECKLIST

- ☐ Terminal launched and root access confirmed
- ☐ Authorized scope confirmed
- ☐ First Docker image (ubuntu) pulled
- ☐ First image scanned with Trivy
- ☐ First scan result reviewed and documented
- ☐ Second Docker image (nginx:1.19.6) pulled
- ☐ Second image scanned with Trivy
- ☐ Second scan result reviewed and documented
- ☐ Severity levels and CVEs reviewed
- ☐ Findings compared between the two images
- ☐ Security implications analyzed
- ☐ Countermeasures reviewed

REVIEW QUESTIONS

- 1. What is a Docker image?
- 2. What is Trivy, and what types of targets can it scan?
- 3. Why might one Docker image show 0 vulnerabilities while another shows hundreds?
- 4. What is a CVE, and why is it useful in a vulnerability report?
- 5. Why is severity categorization important when reviewing scan results?
- 6. Why should base images be updated or re-pulled regularly?
- 7. How does image scanning fit into a CI/CD pipeline?
- 8. Why must vulnerability scanning be performed only against authorized images and targets?

TROUBLESHOOTING

Problem	Possible Cause	Recommended Action
Docker pull fails	No internet connectivity or registry unreachable	Verify network connectivity and retry the pull command
Trivy command not found	Trivy is not installed or not on PATH	Confirm Trivy is installed as part of the lab setup
Trivy scan takes a long time	Vulnerability database update in progress	Allow the database update to complete before the scan runs
Scan result differs from the example	Vulnerability databases are updated continuously	This is expected; document the results observed in your own environment
Permission denied running Docker commands	Terminal not running with sufficient privileges	Confirm the terminal session has root access as instructed
No vulnerabilities reported for an image expected to have some	Local vulnerability database may be outdated	Update Trivy's vulnerability database and re-run the scan

BEST PRACTICES

- Always define the assessment scope before scanning.
- Use official or organization-approved base images where possible.
- Scan images before deploying them to any environment.
- Prioritize remediation by severity and exploitability.
- Re-scan images regularly, not just at build time.
- Keep the scanner's vulnerability database up to date.
- Document findings, including CVEs and severity, professionally.
- Avoid pinning to known-outdated version tags in production.
- Remove unused images and containers from the environment.

LAB SUMMARY

This laboratory introduced the process of assessing Docker image security using Trivy, comparing a well-maintained image against an outdated, vulnerable image to illustrate how base-image choice and patch currency directly affect an organization's exposure. Findings from this stage feed directly into a broader container-security lifecycle.

DISCOVER	SCAN	TRIAGE	REMEDIATE	VERIFY